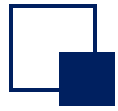




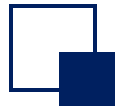
Introduction to Object-Oriented Software Construction



Software is everywhere



Software is everywhere and creates infinite excitement in our lives. In today's information age, the world is becoming more "smart", and the degree of perception, interconnection and intelligence between everything is deepening.



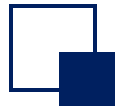
Course content

Introduction to Object-Oriented Software Construction

A way of thinking

A construction method

A course that guides everyone on how to use object-oriented principles to construct software.



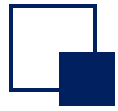
Course content

- Object-Oriented Principles (OOP)
- Java programming language
- Design patterns



Objectives

- OOP Mindset: Thinking in terms of objects and their interactions
- Methodology: Systematic approach to software construction
- Java Proficiency: Mastery of Java language and libraries
- Design Patterns: Understanding proven software design solutions
- Independent Project: Building a complete software system on your own

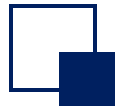


Teaching methods

- Tutorials (48 hours)
 - Teach basic knowledge and ideas
 - In-class discussions, practices, and Q&A
 - Provide relevant reading materials
- Experiments (16 hours)
 - Use the object-oriented programming methods to construct software
 - Contains 4 experiments
 - Complete it independently according to the requirements of the experimental instructions

Example – Aircraft War





Textbooks

- Horstmann, Cay S. *Core java, volume I: fundamentals*. Pearson Education
- Horstmann, Cay S. *Core Java, Volume II: Advanced Features*. Addison-Wesley Professional
- Gamma, Erich, et al. *Design patterns: elements of reusable object-oriented software*. Pearson Deutschland GmbH
- Eckel, Bruce. *Thinking in JAVA*. Prentice Hall Professional



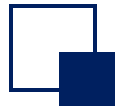
Course and grades

- Tutorials & Experiments
- Final grades:
 - 10% homework and assessment
 - 40% experiments
 - 50% final exam



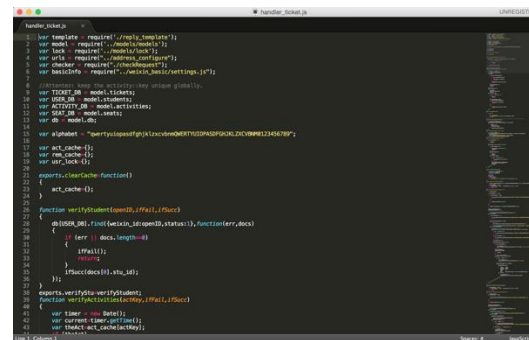
Chapter I

- Software construction basic process and goals
- Object-oriented thinking
- Design patterns
- Introduction to the Java language
- Java development environment construction and program examples



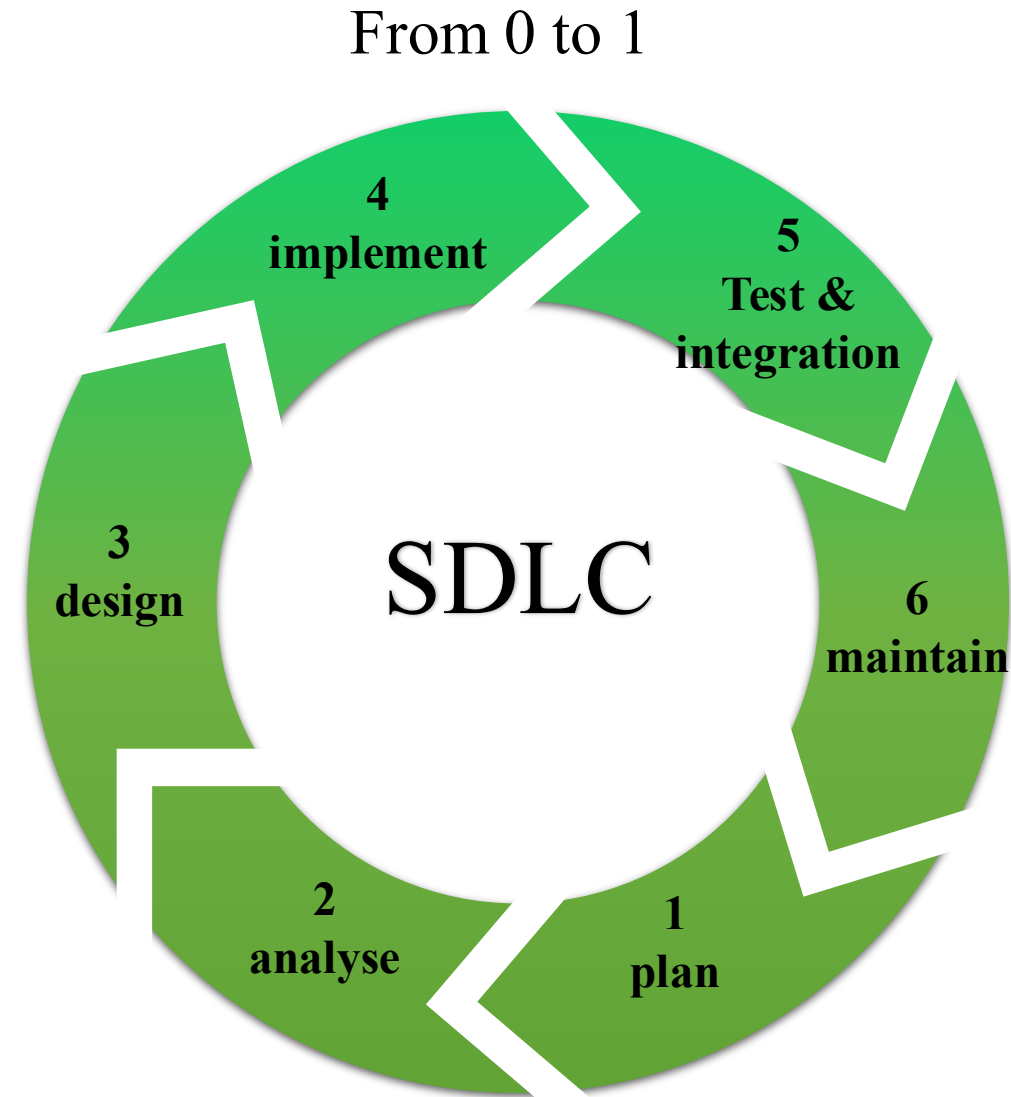
What is software?

- Software = program + data + documentation
 - Program: A set of instructions that the computer can accept and the runtime can provide the required functionality and performance.
 - Data: Data structures that enable programs to manipulate information appropriately.
 - Documentation: Describing the development process, methods and graphic materials used in the program.





Software Development Life Cycle (SDLC)

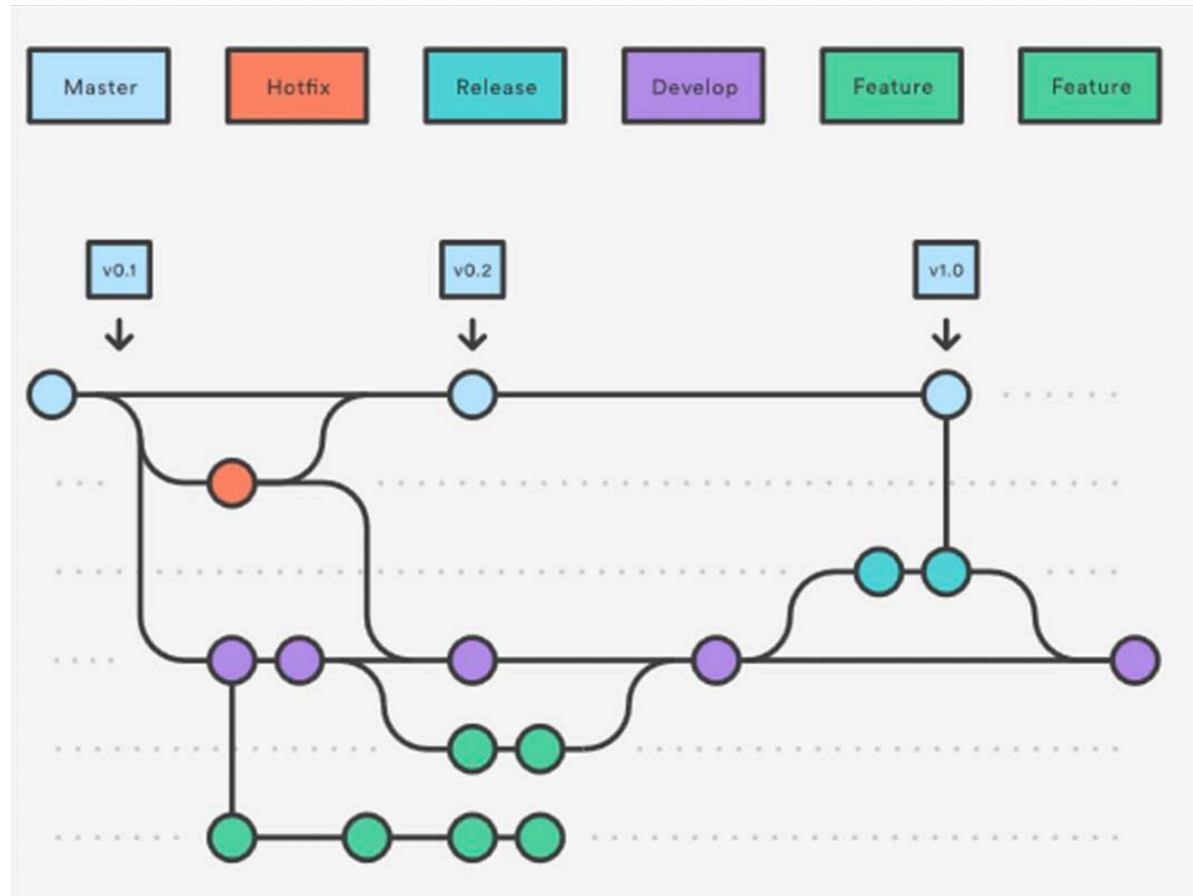




The basic process of software construction

Multiple versions of iterations in software development life

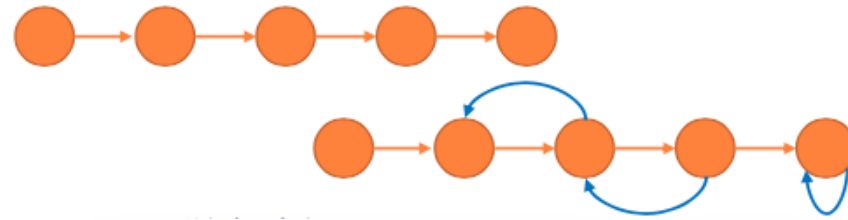
From 1 to n :

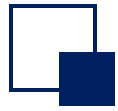




Traditional software development models

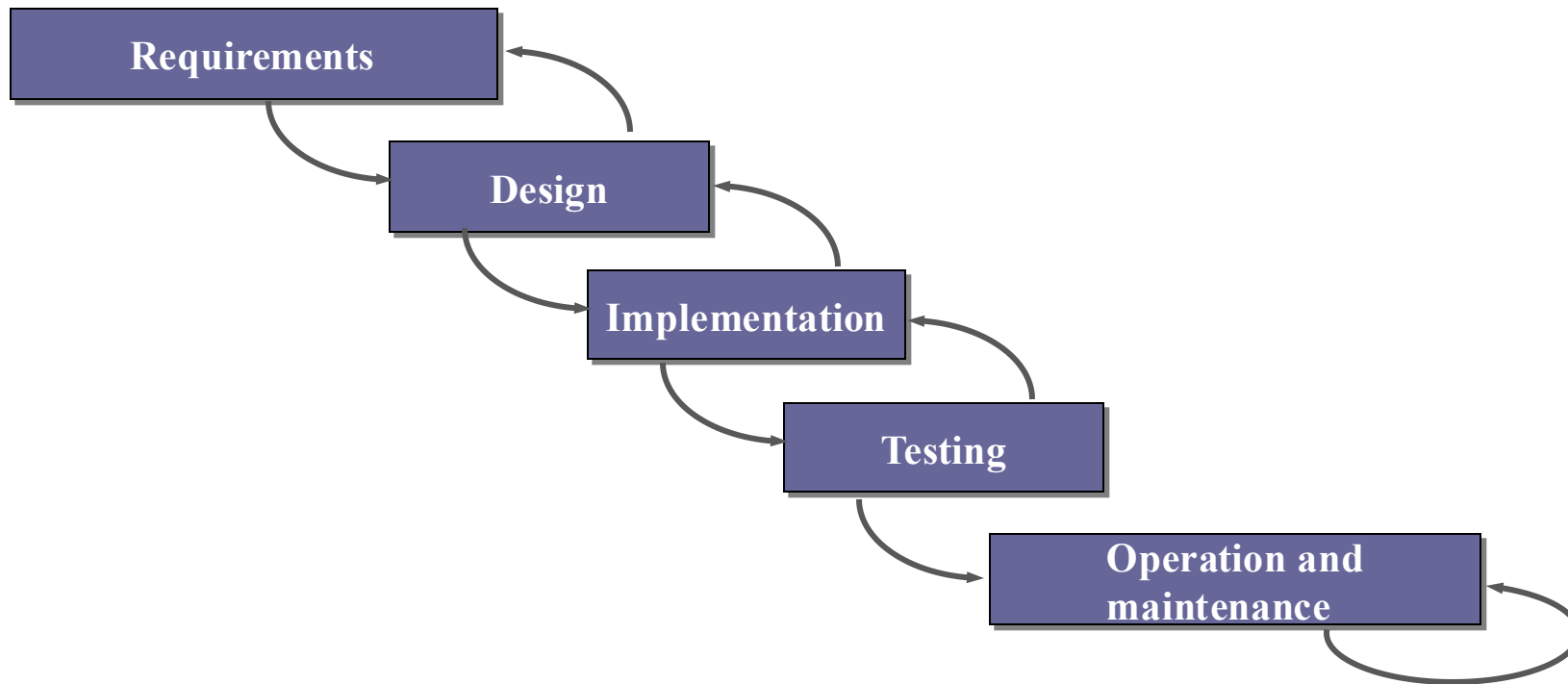
- Two basic models
 - Linear model
 - Iterative process
- Traditional models
 - Waterfall process
 - Incremental process
 - Prototyping process
- Popular models
 - Agile development
 - Test-driven development





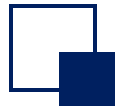
Waterfall process

The waterfall model is the most typical predictive development method, which strictly follows the pre-planned step sequence of requirements analysis, design, coding, integration, testing, and maintenance.



□ Characteristics of the waterfall process

- Linear progression
- The stages are clearly divided
- Overall promotion
- Simple to manage
- Inability to adapt to increases/changes in demand



Waterfall process

For example: aircraft wars

❑1. Requirements

- Character setting
- Game rules
- interface and sound requirements
- Game difficulty design
- Identify needs
- Write a requirements analysis report

❑2. Software design

- Overall architecture design
- Modular design
- Design of classes and interfaces

❑3. Software implementation

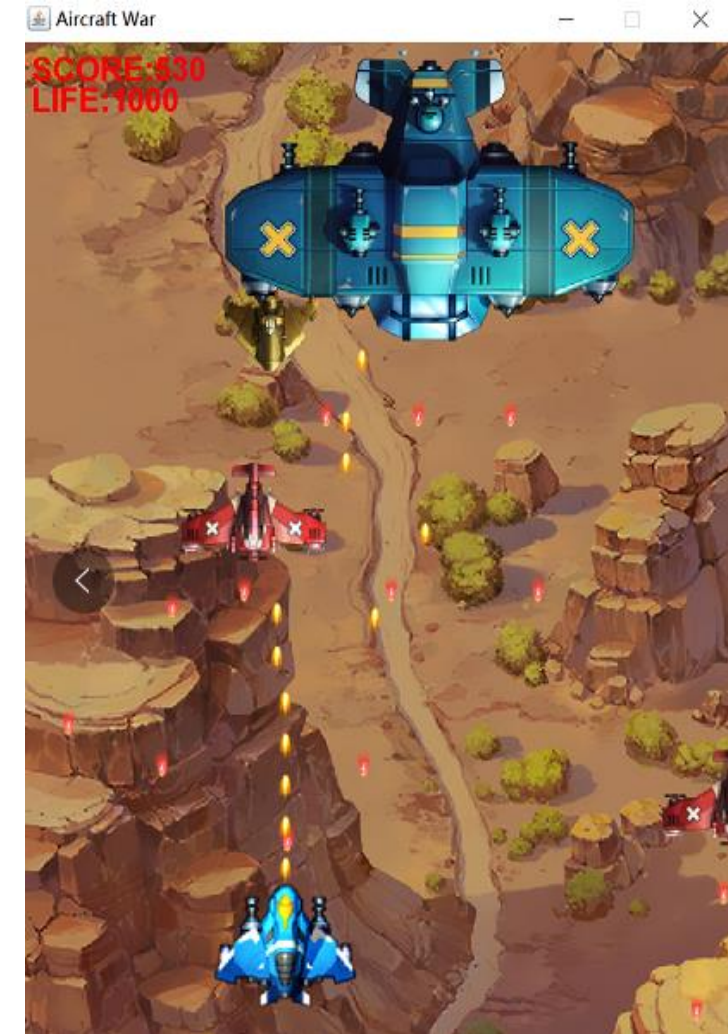
- Programmers write code

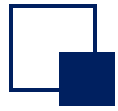
❑4. Software testing

- Test software functions
- correctness
- Black box test, white box test

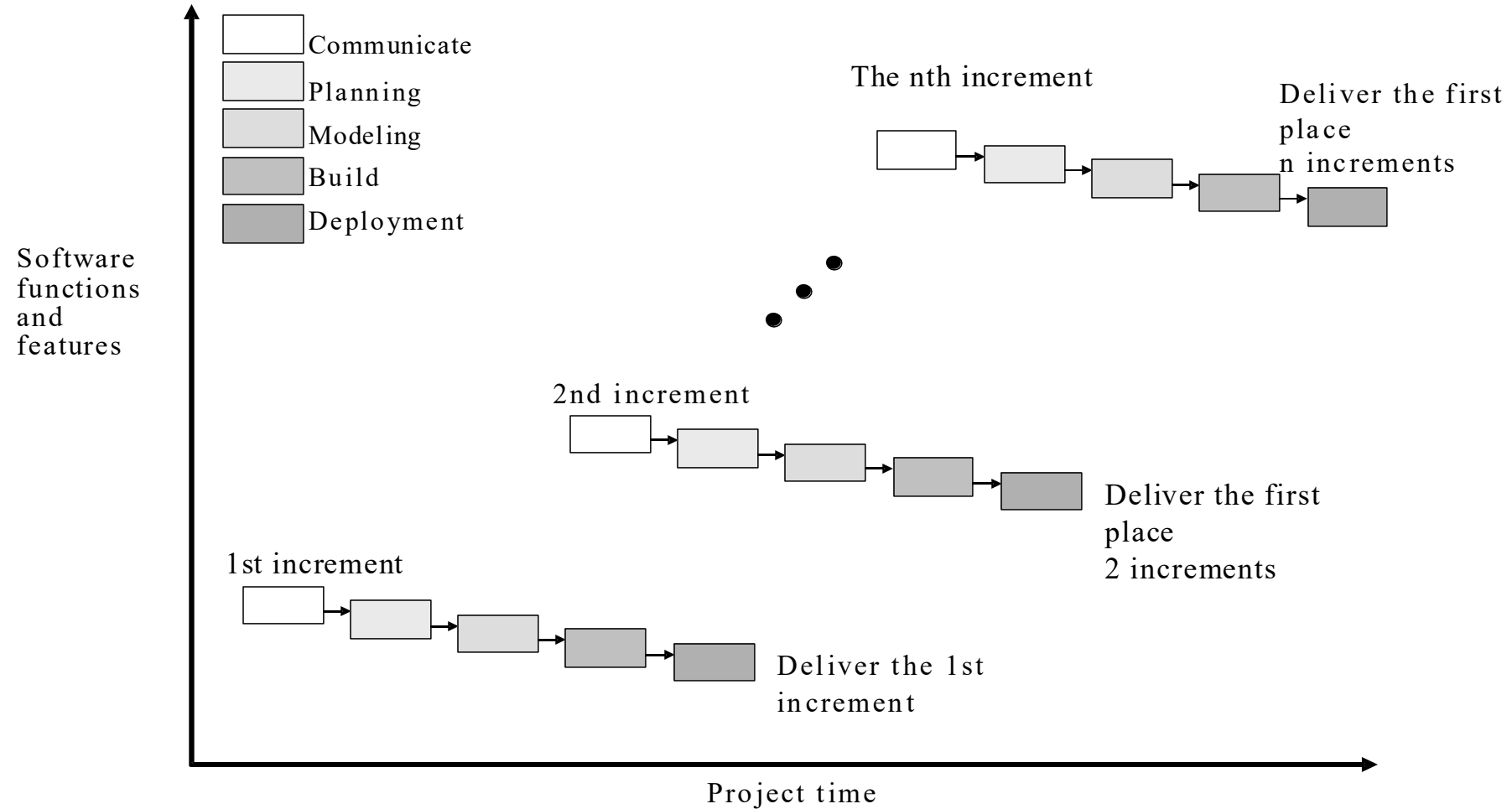
❑5. Software operation and maintenance

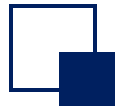
- The software is launched
- Version update





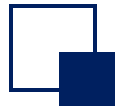
Incremental process





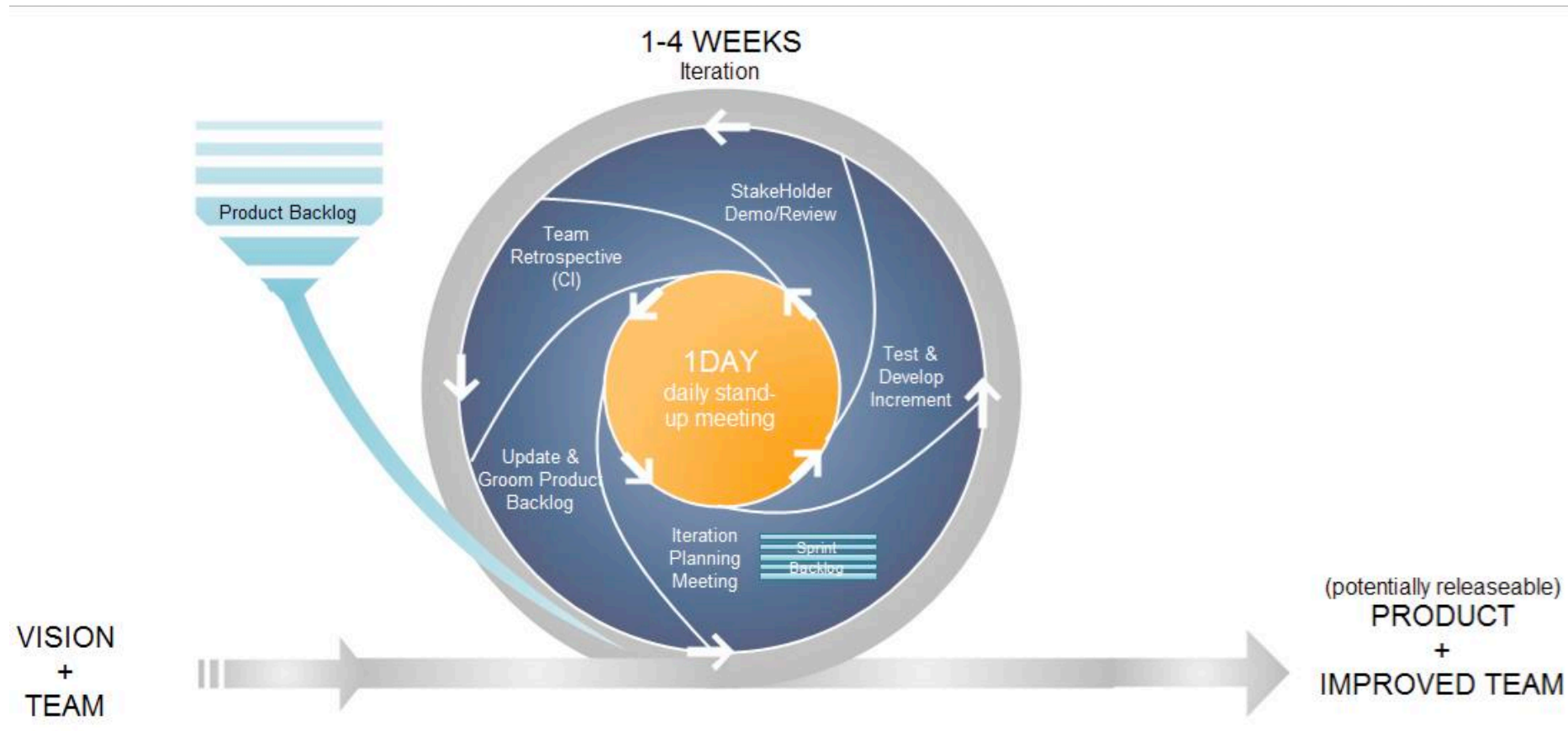
Prototyping process

- The prototyping process is a program that can be quickly established and can run on a computer, and the functions it can accomplish are often a subset of what the final product can accomplish. The role of prototypes in development:
 - (1) Obtain the real needs of users
 - (2) It can be used to determine the technical, cost, and schedule possibilities for a project or parts of a project
- Iterative techniques are employed as prototype systems are continuously adapted to meet the needs of various stakeholders, while also allowing developers to gradually understand the needs of users.



Agile development

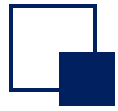
Agile Development: Adapt quickly to changes through rapid iterations and small-scale continuous improvements.





Test-driven development

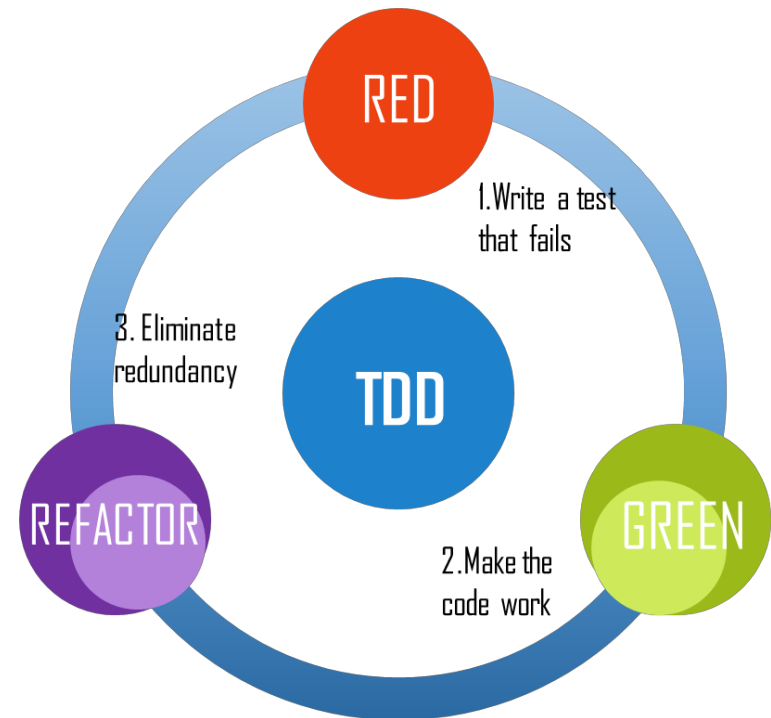
Test-Driven Development (TDD) is a new development method that is different from the traditional software development process. It requires that **the test code be written before writing the code for a function**, and then only the functional code that passes the test is written, and the test is used to promote the entire development. This helps to write concise, usable and high-quality code and accelerates the development process.



Test-driven development

□ Basic Processes (**Red-Green-Refactor**)

- 1) Clarify the current functional requirements and formulate a TODO test list;
- 2) Quickly complete the writing of test cases for this function requirements;
- 3) Test Code Compilation Fails (**RED**);
- 4) Write the corresponding function code;
- 5) Test passed (**GREEN**);
- 6) Refactor the code (**REFACTOR**);
- 7) Loop through the development of all functions.

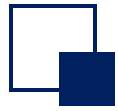




Test-driven development

□ advantage

- Strengthening the understanding of requirements helps to design easy-to-use interfaces from the user's perspective.
- Strengthening the concept of modules and interfaces has led us to implement loosely coupled designs and rely more on interfaces to improve the scalability and resilience of the system.
- Run unit tests frequently to detect errors early and greatly reduce the cost of subsequent testing and fixing.
- Continuous regression testing is provided, and the code is easy to maintain and refactor.
- Test code that verifies functional correctness in real time is the best code documentation and is synchronized with the latest code.
- Reduce stress, reduce worry, increase confidence in our code, and improve program development efficiency.



Test-driven development

Template program

- The main interface of the game
- Hero planes, ordinary enemy planes
- The plane moves
- Hero machine bullet fire
- Crash detection

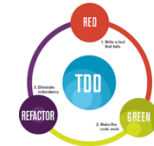
Experiment 1

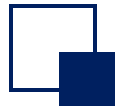
- Added all enemy planes and item types
- Build elite enemy aircraft
- Elite enemy aircraft bullet launch
- The health booster takes effect
- Refactoring code



Experiment 2

- Build a hero machine in singleton mode
- Craft enemy planes and items in factory mode
- Refactoring code





The goal of the software construction

- **Comprehensibility**

- Follow specific naming conventions
- There are sufficient notes and explanations
- The code layout is reasonable:
indent/blank/chunking/aligning, *etc*
- The document is organized in a reasonable way
- The method should not be too long or too short, and
should be easy to understand
- Avoid multi-layer nesting - increase complexity

- **Maintainability**

- When software changes, can you adapt to the change at
a small cost?
- Modular programming, high cohesion and low
coupling
- Use design patterns flexibly

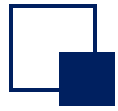
Example #a:

$$z = ((3*x^2) + (4*x) - 5) - ((2*y^2) - (7*y) + 11) / ((3*x^2) + (4*x) - 5)$$

Example #b:

$$\begin{aligned} a &= ((3*x^2) + (4*x) - 5) \\ b &= ((2*y^2) - (7*y) + 11) \\ z &= (a - b) / a \end{aligned}$$





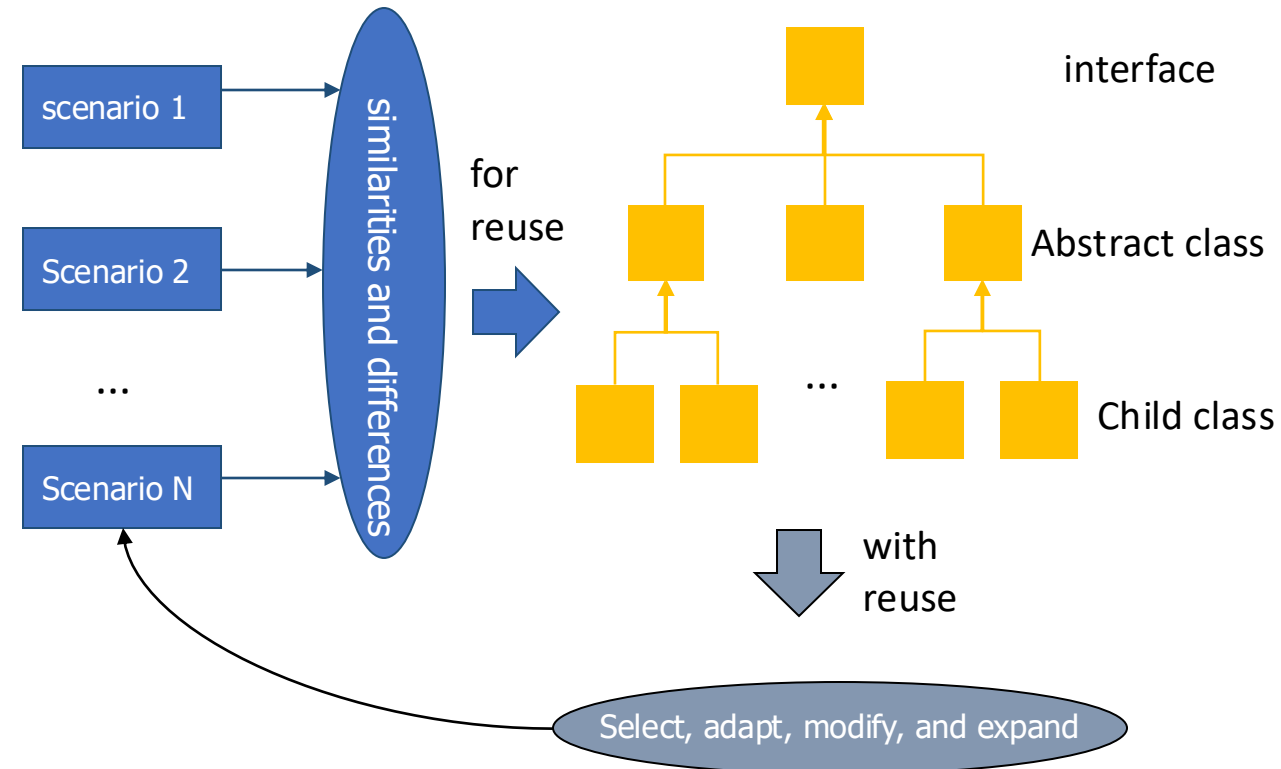
The goal of the software construction

- **Reusability**

- Construct software modules/APIs that can be reused in different applications
- Modular, generic, abstract
- Provide complete fine-grained operations to ensure functional integrity and reuse different operations in different scenarios

- **Spatiotemporal performance**

- Memory performance
- Algorithm performance
- I/O performance



Which of the following is not an advantage of test-driven development?

- A It is conducive to designing interfaces that are easy to use
- B It is suitable for rapid iterative development and flexibly handles changes in requirements
- C Easy to refactor, good for code maintenance
- D Frequent testing can detect errors early

Which of the following is not an advantage of test-driven development?

- ☐ A It is conducive to designing interfaces that are easy to use
- ☒ B It is suitable for rapid iterative development and flexibly handles changes in requirements
- ☐ C Easy to refactor, good for code maintenance
- ☐ D Frequent testing can detect errors early



Chapter I

- Software construction basic process and goals
- **Object-oriented thinking**
- Design patterns
- Introduction to the Java language
- Java development environment construction and program examples



Software engineering methodology

- **Structured Approach (Process-Oriented)**

- Complex world → complex process (the occurrence and development of things)
- Design a set of features (or algorithms) to solve a problem
- Find the right way to store and process data

- **Object-oriented approach**

- Any system is made up of objects that can accomplish a set of related tasks
- If an object depends on a task that is not for which it is responsible, then it needs to access another object that is responsible for this task (calling the method of another object)
- An object cannot directly manipulate the data inside another object, nor can it give other objects direct access to its own data
- All communication must be called by method

Object-oriented three characteristics

❑ Encapsulation:

- hide the properties and implementation details of the object, and only publicly access the method;
- Enhanced security and simplified programming

student

- Private information and operations
- Open action

❑ Inheritance

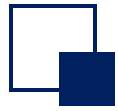
- Subclasses inherit the characteristics and behaviors of their parents
- Implement code reuse

- Students - CS students

❑ Polymorphism

- The ability of the same behavior to have multiple different manifestations ("one interface, multiple methods")
- Improved program scalability and maintainability

- Course:
 - School of Computer Science - Compilation Principles
 - School of Science - Physics



Object-oriented thinking vs structured programming thinking

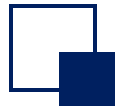
Example: Go to Shenzhen North Railway Station

- Option 1: Drive yourself
 - 1. Get in the car and start
 - 2. Turn left at the school gate and stay on Immortal Avenue
 - 3. If there is a red light, wait, otherwise continue
 - 4. Turn right at the intersection of Zhiyuan Middle Road
 - 5....



The above is a typical thought process of structured programming

The structured method inherits the traditional programming ideas and programming methods to realize the computing function of the computer.



Object-oriented thinking vs structured programming thinking

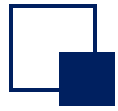
Example: Go to Shenzhen North Railway Station

- Option 2: Take a taxi
 - 1. Call a taxi with software (create a taxi object)
 - 2. Tell the driver: I want to go to Shenzhen North Railway Station (call the target method)
 - 3. Close your eyes and recuperate, and wait quietly to arrive



Object-oriented thinking: everything is an encapsulation of objects and methods.

Object-oriented thinking is premised on simulating things in the objective world and the connections between things.



Object-oriented thinking vs structured programming thinking

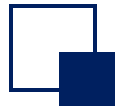
Example: Lead a basketball game

- Plan 1: I will command
 1. A, attack with the ball.
 2. B, come out to help with pick-and-roll.
 3. C, running open seats.
 4. A passes to D
 5. D shoots, and B grabs rebounds
 6.



Structured thinking

The process of implementing a program or algorithm



Object-oriented thinking vs structured programming thinking

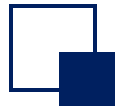
Example: Lead a basketball game

- Plan 2: Arrange troops
 1. A team member as a defender (create object)
 2. B team members are the center (create objects).
 3. ...
 4. The game is played
 5. C is injured, F replaces him (destruction and creation of objects)



Object-oriented thinking

Everything is an encapsulation of objects, methods



Object-oriented thinking vs structured programming thinking

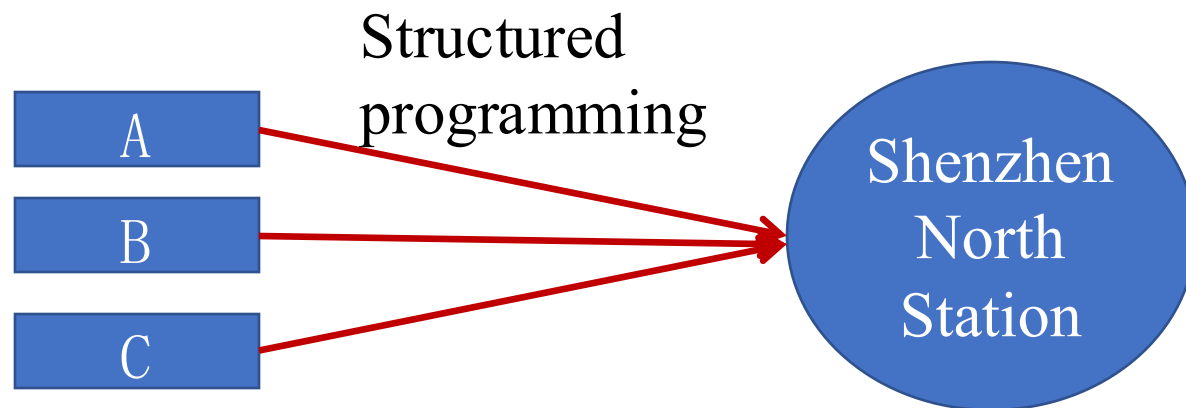
- **Object-oriented basic ideas**

- Point 1: Everything is an object, and the object has properties and methods. Complex objects can be constructed in some way from relatively simple objects.
- Point 2: Discovering the similarity between objects through analogy, that is, the common properties between objects, is the basis for forming object classes.
- Point 3: Interconnection between objects is done by passing "messages". Through message communication between objects, objects are driven to perform a series of operations to complete a task.

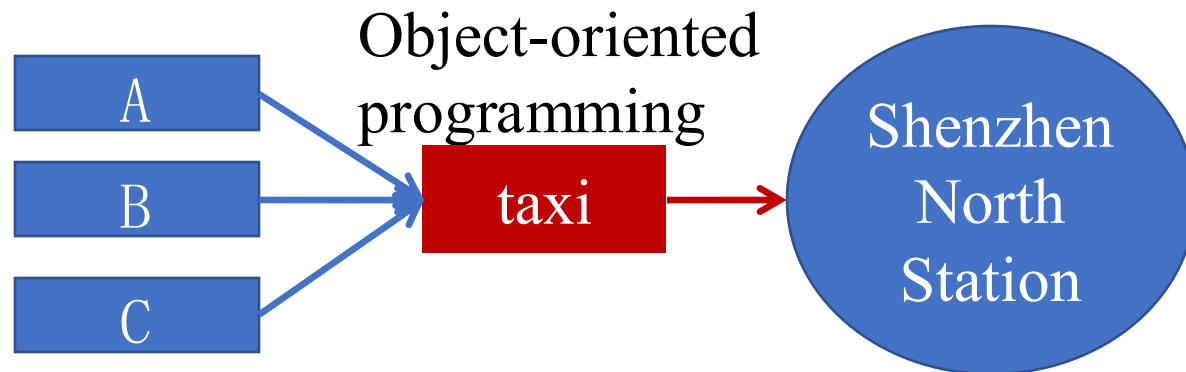
- **Object-oriented advantages:** Modularity, naturalness, concurrency, reusability

- The object-oriented approach makes the software have a good architecture, easy software componentization, software reuse, good scalability and maintenance, and has a high degree of abstraction, so it has high production efficiency.

Advantages of OOP: decoupling, encapsulation



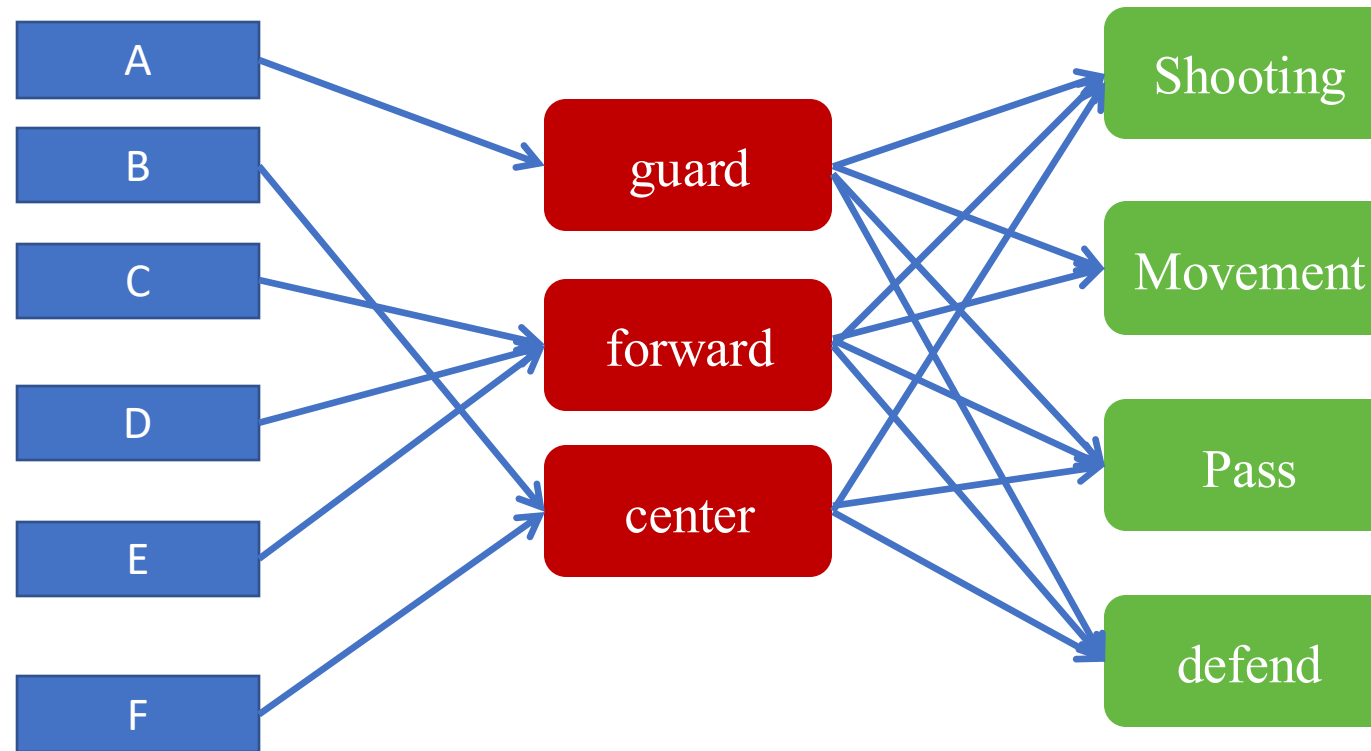
In structured programming, everyone should include the method of getting to Shenzhen North Railway Station if there is a change, such as "Liuxian Avenue road construction". You need to change everyone's program code.



In object-oriented programming, the process of going to the North Station is "encapsulated" separately by creating an object "taxi". It realizes the "decoupling" of ABC's method to Shenzhen North Railway Station. If you want to change it, just change the method of the taxi class.

Advantages of OOP: reuse

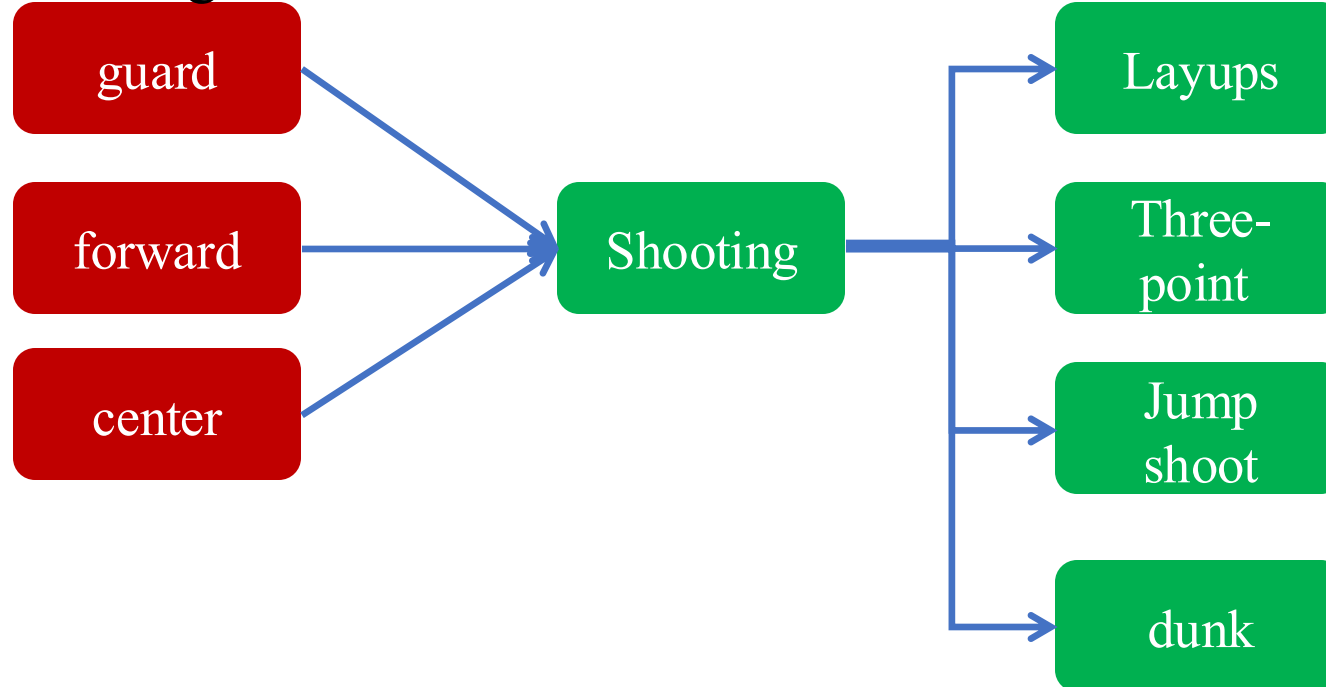
- Structured programming: Each new player is constructed, and his series of actions such as shooting, running, passing, and defending are coded.
- Object-oriented programming: Reuse the previous code modules according to the "class" to which the new team member belongs. You can even quickly create new player types.



Advantages of OOP: Polymorphism

Polymorphism in object-oriented programming allows for different implementations of the same interface, with the advantage of separating the implementation details from the upper objects.

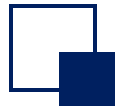
Imagine that a coach on the basketball court only needs to shout "shoot" to the team members, and they can execute this "shooting" instruction in a pre-designed way and with a variety of different strategies.





Chapter I

- Software construction basic process and goals
- Object-oriented thinking
- **Design patterns**
- Introduction to the Java language
- Java development environment construction and program examples



The history of the design pattern

"Each pattern describes a problem that keeps happening around us, and the core of the solution to that problem. This way, you can use the solution again and again without having to do repeated labor. "

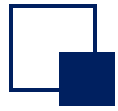
-- Christopher Alexander

Christopher Alexander, a famous American architect and director of the Center for Environmental Structure at the University of California, Berkeley, wrote "A Pattern Language: Towns Building Construction"



The core of the pattern:

Provide solutions to similar problems



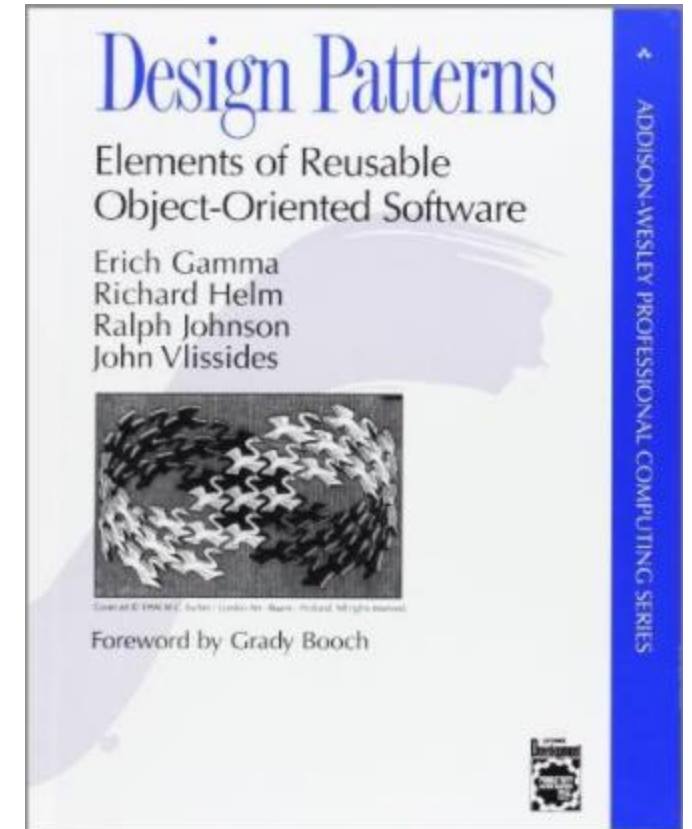
The history of the design pattern

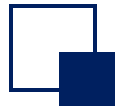
"Patterns" originated in architecture, but are applicable to object-oriented software design.

In 1995, Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides co-authored a book called *Design Patterns - Elements of Reusable Object-Oriented Software*.

The book contains 23 design patterns, and the concept of design patterns in software development is mentioned for the first time. This is a milestone event in the field of design patterns, leading to breakthroughs in software design patterns.

The four authors are collectively known as Gang of Four (GOF).





The concept of design patterns

- Design Pattern

- It is a summary of the design experience of code that has been used repeatedly and has been classified and cataloged. It describes some of the recurring problems in the software design process, as well as the core solution to that problem.
- The purpose is to improve code reusability, code readability, and code reliability。

□ The essence of the design pattern is the practical application of object-oriented design principles, and the full understanding of the encapsulation, inheritance and polymorphism of classes, as well as the relationship and combination relationship of classes.

□ note

- An abstract scheme
- Schemas are not specific libraries, macro definitions, template classes



Chapter I

- Software construction basic process and goals
- Object-oriented thinking
- Design patterns
- Introduction to the Java language
- Java development environment construction and program examples



What is Java?

- An object-oriented programming language
 - With pleasing syntax and easy-to-understand semantics
- A high-quality execution environment
 - Safety, portability, automatic garbage collection
- A huge library
 - Cool drawing function, network connection function, database access function





History of Java Language

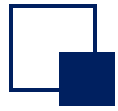
In 1991, a group of engineers at Sun, led by James Gosling, planned to design a language with short, compact code and platform-agnostic. This project was named "Green".



James Gosling



Green project development
team



History of Java Language

version	year	New language features	# of classes and interfaces
1.0	1996	The language itself	211
1.1	1997	Inner class	477
1.2	1998	strictfp Modifiers	1524
1.3	2000		1840
1.4	2002	assertion	2723
5.0	2004	Generic types, "for each" loops, variable element parameters, Automatic boxing, metadata, enumeration, static import	3279
6	2006		3793
7	2011	String-based selection statements, diamond operators, Binary literal volume and exception handling are enhanced	4024
8	2014	lambda expressions, interfaces containing default methods, Stream and date/time library	4240
9	2017	modules, other languages, and libraries have been enhanced	6005



The Java language is one of the most popular languages

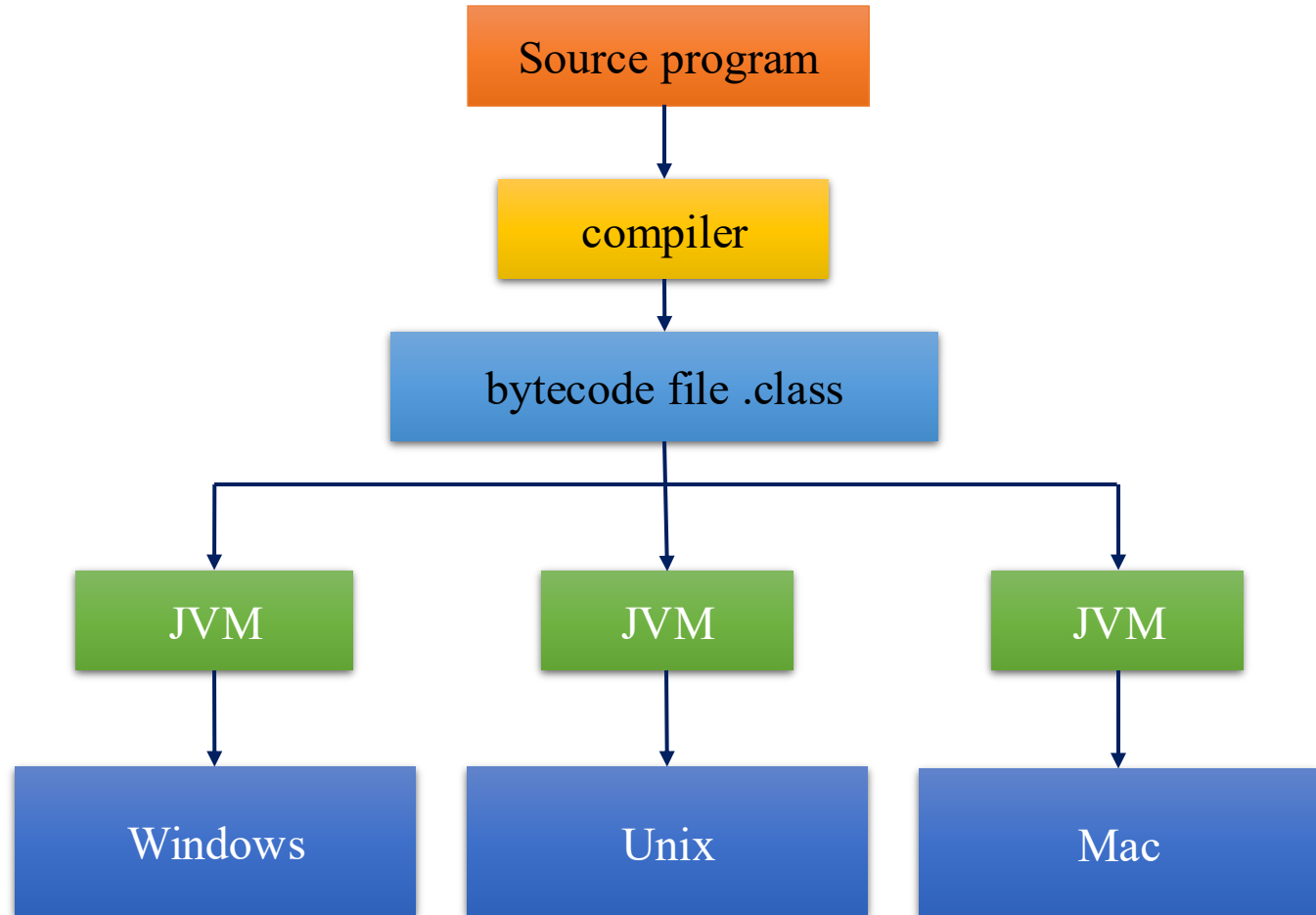
Language Ranking: IEEE Spectrum

Rank	Language	Type	Score
1	Python▼	  	100.0
2	Java▼	  	95.3
3	C▼	  	94.6
4	C++▼	  	87.0
5	JavaScript▼		79.5

<https://spectrum.ieee.org/static/interactive-the-top-programming-languages-2020>

Java language features

- Simplicity
- Object-oriented
- Security
- Platform agnostic →
- Multithreading
- Network functions
- Execution efficiency





The main differences between Java and C/C++

- Cross-platform
 - C/C++ language is compiled into machine code,
 - Java language is compiled into bytecode, and machine code is compiled through the JVM.
- Abolition of pointers
 - C/C++ languages have pointer types,
 - Java languages do not.
- inherit
 - C++ allows multiple inheritance,
 - Java language only allows single inheritance.
- speed
 - C/C++ runs fast,
 - Java runs slowly.



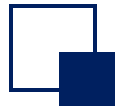
The basic process of running a Java program

- Write Java source programs using the editor
 - Source file name: The main class name .java
- Use the compiler (javac) to compile the source program into bytecode
 - Bytecode file name: source file name .class
- Run with a virtual machine (interpreter, java).
 - Running process: loading, code verification, interpretation and execution
 - For example, the java source filename .class



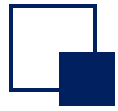
Chapter I

- Software construction basic process and goals
- Object-oriented thinking
- Design patterns
- Introduction to the Java language
- Java development environment construction and program examples



Java Development Kit (JDK)

- The JDK is a software development kit for the Java language that contains a set of components for Java development
 - Javac: compiler that compiles source code with the suffix .java into bytecode with the suffix .class
 - Java: Run the tool, run the bytecode of the .class
- Select the appropriate JDK according to the operating system version for installation
 - <https://www.oracle.com/java/technologies/javase-jdk11-downloads.html>
- After installation, you need to configure two environment variables, JAVA_HOME and PATH
 - JAVA_HOME indicate the JDK installation path
 - PATH allows the system to recognize java commands in any path



Program example – using command-line tools

- Once the JDK is installed, you can compile and run Java programs from the command line.
 - Create sample code
 - Open a terminal window
 - Go to the sample code directory
 - Type the following command:
 - javac Welcome.java
 - java Welcome

Welcome/ Welcome.java

```
public class Welcome
{
    public static void main(String[] args)
    {
        String greeting = "Welcome to Java!";
        System.out.println(greeting);
        for (int i = 0; i < greeting.length(); i++)
            System.out.print("=");
        System.out.println();
    }
}
```

```
管理员: Windows PowerShell
Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。
尝试新的跨平台 PowerShell https://aka.ms/pscore6

PS C:\Windows\system32> cd D:\Hitsz\Welcome
PS D:\Hitsz\Welcome> javac Welcome.java
PS D:\Hitsz\Welcome> java Welcome
Welcome to Java!
```



Java integrated development environment

- IntelliJ IDEA、Eclipse、NetBeans

```
File Edit View Navigate Code Analyze Refactor Build Run Tools VCS Window Welcom
Welcome > src > Welcome
Project
Welcome D:\Hitsz\Welcome
  .idea
  out
  src
    Welcome
    Welcome.iml
    Welcome.java
  External Libraries
  Scratches and Consoles
Welcome.java x
1 public class Welcome {
2     public static void main(String[] args)
3     {
4         String greeting = "Welcome to Java!";
5         System.out.println(greeting);
6         for (int i = 0; i < greeting.length(); i++)
7             System.out.print("=");
8         System.out.println();
9     }
10 }
Run: Welcome x
"C:\Program Files\Java\jdk-11.0.9\bin\java.exe" "-javaagent:C:\Program Files\JetBrains
Welcome to Java!
=====
Process finished with exit code 0
Run TODO Problems Terminal Build Event Log
Build completed successfully in 2 sec, 614 ms (a minute ago) 6:1 CRLF UTF-8 4 spaces
```


The three main characteristics of OOP are:

- A encapsulation
- B inheritance
- C modularization
- D Polymorphism
- E Streamline

The three main characteristics of OOP are:

- ☒ A encapsulation
- ☒ B inheritance
- ☐ C modularization
- ☒ D Polymorphism
- ☐ E Streamline



Chapter I

- Software construction basic process and goals
- Object-oriented thinking
- Design patterns
- Introduction to the Java language
- Java development environment construction and program examples